Machine Learning • Bab 5

5.11 dan 5.12 Implementasi Python Decision Tree & Pruning

Modul lengkap: Praktikum 5.1 – 5.6 dan Latihan Praktikum 5.12,
dengan penjelasan tiap langkah, output asli, dan interpretasi

Disusun oleh

Ahmad Dandi Subhani

202343500126

Buka Notebook di Google Colab

5.11. Implementasi Python

Bagian ini mempraktikkan algoritma decision tree dengan tiga skenario: tanpa pruning (tumbuh penuh), pre-pruning, dan post-pruning (cost complexity pruning). Dataset yang dipakai adalah Breast Cancer Wisconsin dari sklearn.datasets (569 pasien, 30 fitur numerik, 2 kelas: malignant dan benign).

Praktikum 5.1 Persiapan Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, validation_curve
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
```

Penjelasan: numpy dan pandas untuk olah data, matplotlib untuk visualisasi. Dari sklearn diimpor load_breast_cancer (dataset), train_test_split (split data), DecisionTreeClassifier (model pohon keputusan), plot_tree (visualisasi pohon), serta accuracy_score untuk evaluasi.

Praktikum 5.2 Import Dataset

```
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
print("Ukuran data:", X.shape)
print("Distribusi kelas:", np.bincount(y))
```

```
Ukuran data: (569, 30)
Distribusi kelas: [212 357]
```

Penjelasan: dataset dimuat sebagai DataFrame (as_frame=True) agar nama fiturnya terbaca. X berisi 30 fitur, y berisi label (0 = malignant sebanyak 212, 1 = benign sebanyak 357). Ketidakseimbangan ini masih wajar untuk klasifikasi biner, tetapi proporsinya nanti dijaga saat split data.

Praktikum 5.3 Preprocessing (Split Data)

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)
```

Penjelasan: data dibagi 80% latih dan 20% uji. stratify=y menjaga proporsi kedua kelas tetap sama di data latih dan uji, sedangkan random_state=42 membuat pembagiannya dapat diulang (reproducible).

Praktikum 5.4 Training Model (dengan dan tanpa Pruning)

```
# Model tanpa pruning (tumbuh penuh)
tree_full = DecisionTreeClassifier(random_state=42)
tree_full.fit(X_train, y_train)

# Model dengan pre-pruning
tree_pruned = DecisionTreeClassifier(max_depth=4,
    min_samples_leaf=5, random_state=42)
tree_pruned.fit(X_train, y_train)

# Model dengan post-pruning (Cost Complexity Pruning)
path = tree_full.cost_complexity_pruning_path(X_train, y_train)
ccp_alpha_terpilih = path.ccp_alphas[len(path.ccp_alphas)//2]  # contoh pemilihan alpha
print("ccp_alpha terpilih:", ccp_alpha_terpilih)
tree_ccp = DecisionTreeClassifier(ccp_alpha=ccp_alpha_terpilih,
    random_state=42)
tree_ccp.fit(X_train, y_train)
```

```
ccp_alpha terpilih: 0.005274725274725275

DecisionTreeClassifier(ccp_alpha=np.float64(0.005274725274725275),
                       random_state=42)
```

Penjelasan: tiga model dilatih untuk dibandingkan.

- **Tanpa pruning (tree_full):** pohon dibiarkan tumbuh sampai semua daun murni. Berisiko overfitting karena menghafal data latih.
- **Pre-pruning (tree_pruned):** pertumbuhan dibatasi sejak awal dengan max_depth=4 (kedalaman maksimal 4) dan min_samples_leaf=5 (setiap daun minimal 5 sampel).
- **Post-pruning (tree_ccp):** pohon ditumbuhkan penuh dulu, lalu cabang-cabang yang lemah dipangkas berdasarkan cost complexity pruning path. Nilai ccp_alpha diambil dari tengah pruning path sebagai contoh (nilai terpilih: 0,005275).

Praktikum 5.5 Evaluasi

```
for nama, model in [("Tanpa Pruning", tree_full),
    ("Pre-Pruning", tree_pruned), ("Post-Pruning (CCP)", tree_ccp)]:
    acc_train = accuracy_score(y_train, model.predict(X_train))
    acc_test = accuracy_score(y_test, model.predict(X_test))
    print(f"{nama}: Akurasi Train={acc_train:.3f}, Akurasi Test={acc_test:.3f}, Gap={acc_train-
acc_test:.3f}")
```

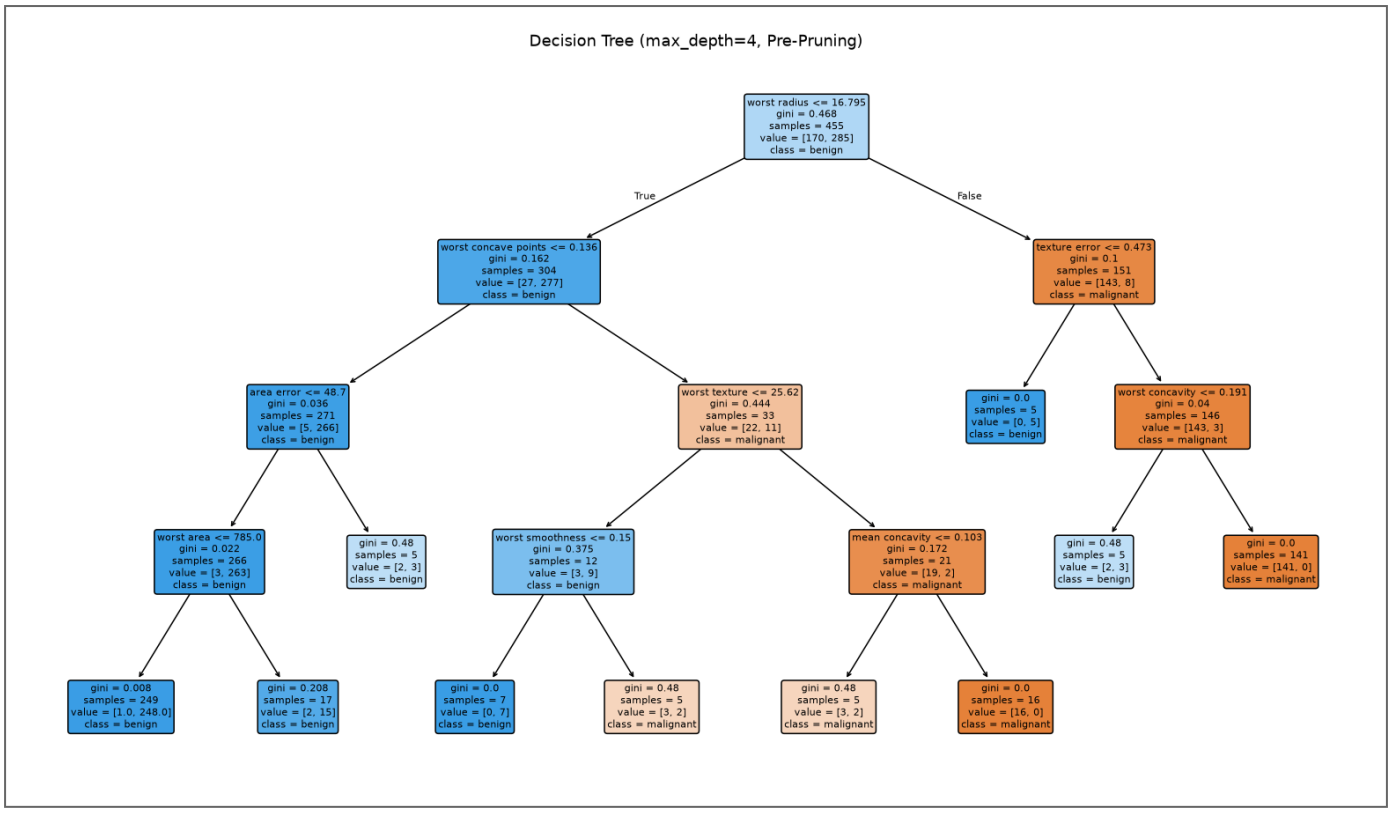
```
Tanpa Pruning: Akurasi Train=1.000, Akurasi Test=0.912, Gap=0.088
Pre-Pruning: Akurasi Train=0.976, Akurasi Test=0.921, Gap=0.055
Post-Pruning (CCP): Akurasi Train=0.978, Akurasi Test=0.930, Gap=0.048
```

Penjelasan: setiap model diukur akurasinya pada data latih dan data uji. Gap = selisih keduanya; gap besar menandakan model terlalu menyesuaikan diri dengan data latih (overfitting).

Model	Kedalaman	Jumlah daun	Akurasi train	Akurasi test	Gap
Tanpa pruning	7	19	1,000	0,912	0,088
Pre-pruning	4	10	0,976	0,921	0,055
Post-pruning (CCP)	4	7	0,978	0,930	0,048

Praktikum 5.6 Visualisasi

```
plt.figure(figsize=(14,8))
plot_tree(tree_pruned, feature_names=data.feature_names,
    class_names=data.target_names, filled=True, rounded=True,
    fontsize=7)
plt.title("Decision Tree (max_depth=4, Pre-Pruning)")
plt.show()
```



Gambar 5.1: Visualisasi decision tree dengan pre-pruning (max_depth=4, min_samples_leaf=5).

Penjelasan: pohon hasil pre-pruning digambar dengan plot_tree. Setiap node menampilkan fitur dan nilai ambang pemisah (misalnya worst radius ≤ 16,795), nilai gini impurity, jumlah sampel, dan kelas mayoritas. Warna biru = benign, oranye = malignant; makin pekat warnanya makin murni node tersebut.

5.11.1. Interpretasi

Model tanpa pruning umumnya menunjukkan gap besar antara akurasi train dan test (indikasi overfitting), sedangkan model dengan pre-pruning atau post-pruning menunjukkan gap yang lebih kecil, meskipun akurasi train sedikit menurun. Pemilihan strategi pruning terbaik sebaiknya didasarkan pada validation curve atau cross-validation, bukan sekadar mencoba satu nilai parameter.

Temuan dari hasil praktik

- **Tanpa pruning** mencapai akurasi train sempurna (1,000) tetapi gap terbesar (0,088), pola khas overfitting: pohon sedalam 7 dengan 19 daun menghafal data latih.
- **Pre-pruning** menurunkan gap menjadi 0,055 dengan akurasi test 0,921, pohon hanya sedalam 4 dengan 10 daun, jauh lebih sederhana dan mudah diinterpretasi.
- **Post-pruning (CCP)** memberi hasil terbaik: akurasi test tertinggi (0,930) dengan gap terkecil (0,048) dan pohon paling ringkas (7 daun).

Kesimpulannya, pruning terbukti mengurangi overfitting sekaligus menyederhanakan model. Pada kasus ini post-pruning (CCP) sedikit mengungguli pre-pruning, tetapi pemilihan parameter yang optimal tetap harus melalui validation curve atau cross-validation, bukan satu nilai coba-coba.

5.12. Latihan Praktikum

Latihan 1 Grid Search Parameter Pre-Pruning

Soal: Lakukan grid search manual atas kombinasi max_depth di {2, 4, 6, None} dikali min_samples_leaf di {1, 5, 10}. Cetak akurasi test tiap kombinasi dalam tabel rapi, lalu simpulkan kombinasi terbaiknya.

```
print(f"{'max_depth':>10s} | {'min_samples_leaf':>16s} | {'Akurasi Test':>12s}")
print("-" * 46)

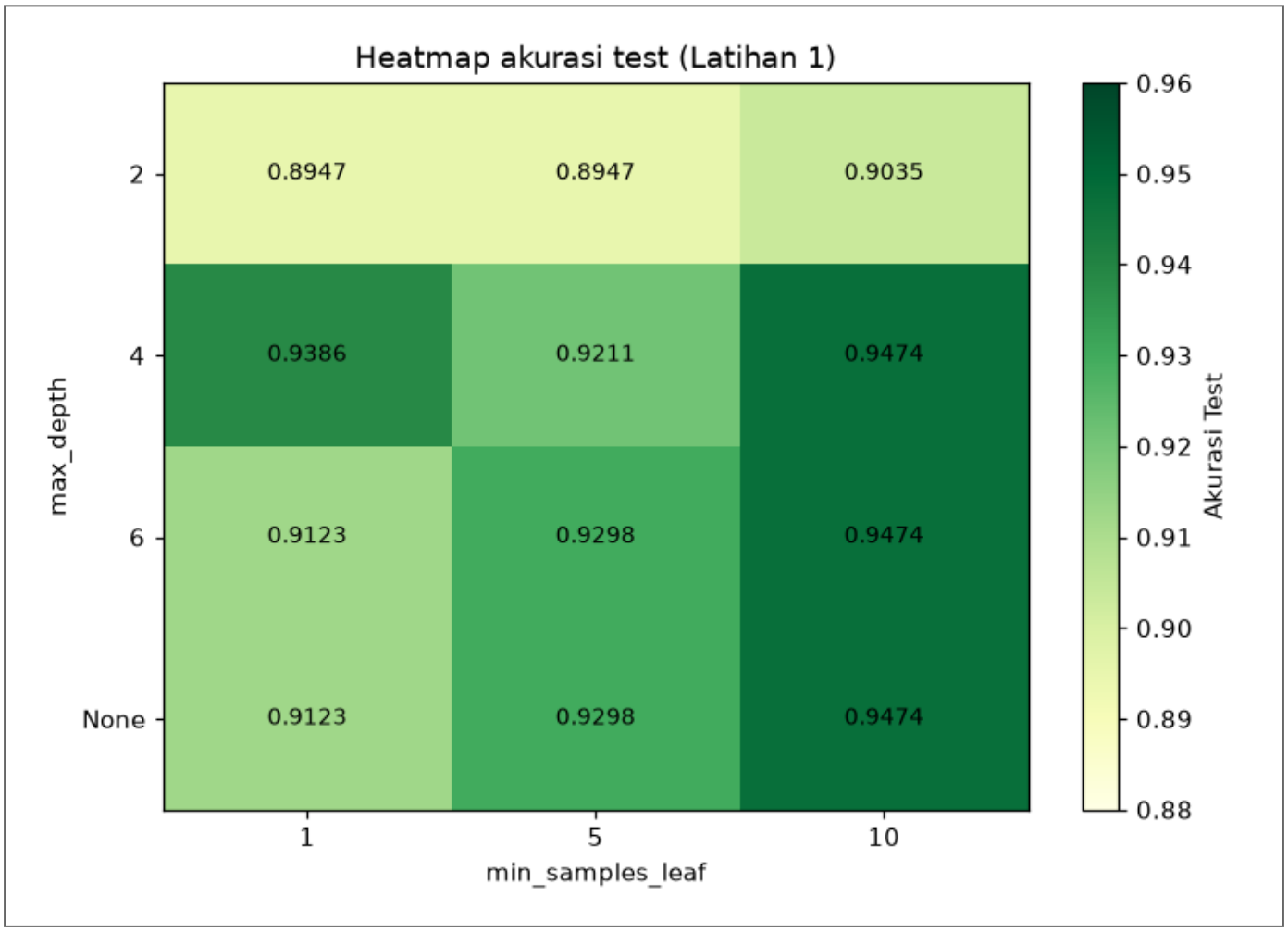
hasil_grid = []
for md in [2, 4, 6, None]:
    for msl in [1, 5, 10]:
        m = DecisionTreeClassifier(max_depth=md, min_samples_leaf=msl, random_state=42)
        m.fit(X_train, y_train)
        ak = accuracy_score(y_test, m.predict(X_test))
        hasil_grid.append((ak, md, msl))
        print(f"{str(md):>10s} | {msl:16d} | {ak:12.4f}")

terbaik = max(hasil_grid, key=lambda t: t[0])
print(f"\nKombinasi terbaik: max_depth={terbaik[1]}, min_samples_leaf={terbaik[2]} "
      f"-> akurasi test {terbaik[0]:.4f}")
```

```
 max_depth | min_samples_leaf | Akurasi Test
-------------------------------------------
         2 |                1 |       0.8947
         2 |                5 |       0.8947
         2 |               10 |       0.9035
         4 |                1 |       0.9386
         4 |                5 |       0.9211
         4 |               10 |       0.9474
         6 |                1 |       0.9123
         6 |                5 |       0.9298
         6 |               10 |       0.9474
      None |                1 |       0.9123
      None |                5 |       0.9298
      None |               10 |       0.9474

Kombinasi terbaik: max_depth=4, min_samples_leaf=10 -> akurasi test 0.9474
```

Interpretasi Latihan 1: dari 12 kombinasi, akurasi test tertinggi adalah **0,9474**, dicapai tiga kombinasi: (4, 10), (6, 10), dan (None, 10). Polanya konsisten: min_samples_leaf=10 selalu lebih baik daripada 1 atau 5 di tiap nilai max_depth, artinya membatasi daun minimum adalah pre-pruning yang efektif di dataset ini. max_depth=None tidak selalu jelek asal min_samples_leaf-nya cukup besar (10): pohon dibiarkan tumbuh tetapi tiap daun harus didukung minimal 10 sampel, sehingga split berbasis noise tetap dicegah. Kombinasi terbaik yang paling sederhana adalah **max_depth=4, min_samples_leaf=10**: dipilih yang paling dangkal di antara yang seri, karena pohon lebih kecil lebih mudah diinterpretasi (prinsip Occam's razor).



Gambar 5.2: Heatmap akurasi test per kombinasi pre-pruning (Latihan 1).

Latihan 2 Cost Complexity Pruning Path

Soal: Untuk tiap alpha di path.ccp_alphas, latih decision tree dan catat akurasi testnya. Plot ccp_alpha vs akurasi test, tentukan alpha optimal, lalu bandingkan jumlah leaf node sebelum dan sesudah pruning.

```
# Hitung pruning path dulu dari data latih
path = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X_train, y_train)
alphas = path.ccp_alphas
ak_test_ccp, n_daun = [], []

for a in alphas:
    m = DecisionTreeClassifier(ccp_alpha=a, random_state=42)
    m.fit(X_train, y_train)
    ak_test_ccp.append(accuracy_score(y_test, m.predict(X_test)))
    n_daun.append(m.get_n_leaves())

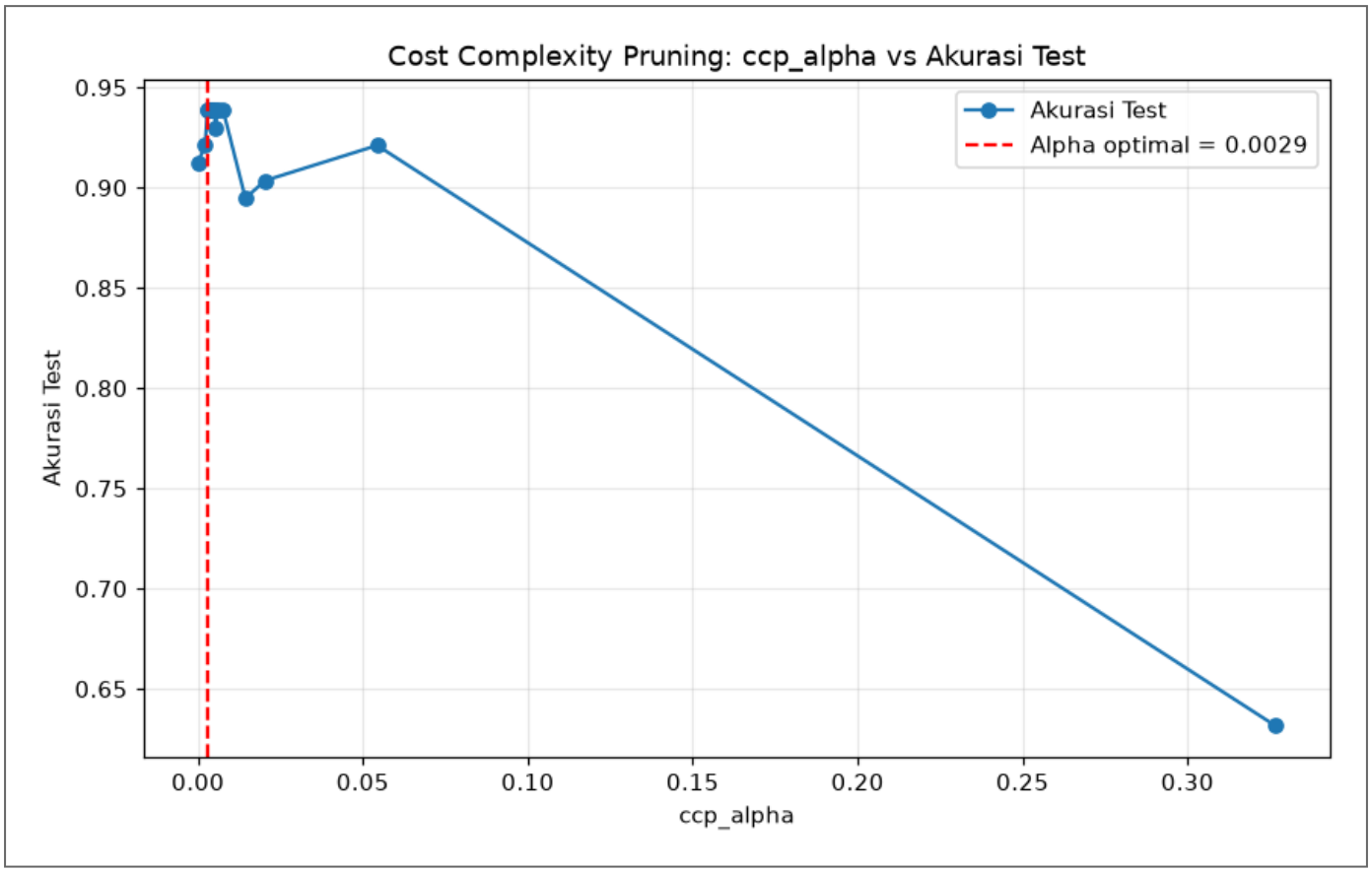
print(f"{'ccp_alpha':>10s} | {'Akurasi Test':>12s} | {'Jumlah Daun':>11s}")
print("-" * 41)
for a, ak, nd in zip(alphas, ak_test_ccp, n_daun):
    print(f"{a:10.6f} | {ak:12.4f} | {nd:11d}")

idx_opt = int(np.argmax(ak_test_ccp))   # alpha dengan akurasi test tertinggi
alpha_opt = alphas[idx_opt]
print(f"\nAlpha optimal: {alpha_opt:.6f} -> akurasi test {ak_test_ccp[idx_opt]:.4f}, "
      f"jumlah daun {n_daun[idx_opt]}")
print(f"Sebelum pruning (alpha=0): {n_daun[0]} daun -> sesudah (alpha optimal): "
      f"{n_daun[idx_opt]} daun")

plt.figure(figsize=(8, 5))
plt.plot(alphas, ak_test_ccp, marker="o", label="Akurasi Test")
plt.axvline(alpha_opt, color="red", linestyle="--",
            label=f"Alpha optimal = {alpha_opt:.4f}")
plt.xlabel("ccp_alpha")
plt.ylabel("Akurasi Test")
plt.title("Cost Complexity Pruning: ccp_alpha vs Akurasi Test")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
 ccp_alpha | Akurasi Test | Jumlah Daun
-----------------------------------------
  0.000000 |       0.9123 |          19
  0.002181 |       0.9211 |          15
  0.002866 |       0.9386 |          12
  0.002930 |       0.9386 |          11
  0.003956 |       0.9386 |          10
  0.004251 |       0.9386 |           9
  0.005024 |       0.9386 |           8
  0.005275 |       0.9298 |           7
  0.005934 |       0.9386 |           6
  0.007641 |       0.9386 |           5
  0.014390 |       0.8947 |           4
  0.020386 |       0.9035 |           3
  0.054334 |       0.9211 |           2
  0.326617 |       0.6316 |           1
```

```
Alpha optimal: 0.002866 -> akurasi test 0.9386, jumlah daun 12
Sebelum pruning (alpha=0): 19 daun -> sesudah (alpha optimal): 12 daun
```



Gambar 5.3: Kurva ccp_alpha vs akurasi test (Latihan 2).

Interpretasi Latihan 2:

- Di alpha=0 (tanpa pruning): 19 daun, akurasi test 0,9123. Begitu alpha dinaikkan, jumlah daun turun bertahap (19 ke 15 ke 12, dan seterusnya): cabang yang lemah dipangkas satu per satu.
- **Alpha optimal = 0,002866** dengan akurasi test **0,9386** dan **12 daun**. Pruning menyusutkan pohon sekitar 37 persen (19 menjadi 12 daun) sambil menaikkan akurasi, bukti bahwa cabang yang dipangkas memang noise.
- Kurvanya menunjukkan pola naik lalu turun: akurasi naik dulu (noise terbuang), lalu **anjlok ke 0,6316** di alpha maksimum (0,3266) ketika pohon tinggal 1 daun. Pruning berlebihan sama buruknya, yaitu underfitting.
- **Kesimpulan:** post-pruning menghasilkan model yang **lebih kecil DAN lebih akurat** (12 daun, 0,9386) dibanding tanpa pruning (19 daun, 0,9123).

Latihan 3 Pilih Pohon yang Sederhana tetapi Tetap Akurat

Soal: Ikuti 5 langkah: (1) muat breast cancer dataset, (2) split stratified 80:20, (3) uji max_depth di {2, 4, 6, None}, (4) uji min_samples_leaf di {1, 5, 10}, (5) bandingkan train, CV, test, dan jumlah leaf. **Output wajib:** tabel parameter, akurasi train dan CV, akurasi test final,

gap train-test, jumlah leaf node, dan visualisasi pohon terpilih. **Aturan main:** hyperparameter dipilih dari skor **CV** (cross-validation 5-fold di data latih saja), bukan dari skor test yang diulang-ulang. Data test hanya dipakai **sekali** di paling akhir.

```
# Langkah 1-4: grid manual + CV (StratifiedKFold 5-fold, hanya di data latih)
from sklearn.model_selection import StratifiedKFold, cross_val_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

hasil = []
for md in [2, 4, 6, None]:
    for msl in [1, 5, 10]:
        m = DecisionTreeClassifier(max_depth=md, min_samples_leaf=msl, random_state=42)
        m.fit(X_train, y_train)
        acc_train = accuracy_score(y_train, m.predict(X_train))
        skor_cv = cross_val_score(m, X_train, y_train, cv=skf)
        hasil.append({"max_depth": md, "min_samples_leaf": msl,
                      "train": acc_train,
                      "cv_mean": skor_cv.mean(), "cv_std": skor_cv.std(),
                      "n_leaf": m.get_n_leaves()})

print(f"{'max_depth':>10s} | {'min_samples_leaf':>16s} | {'train':>7s} | {'CV mean+-std':>14s} |
{'n_leaf':>7s}")
print("-" * 64)
for r in hasil:
    print(f"{str(r['max_depth']):>10s} | {r['min_samples_leaf']:16d} | "
          f"{r['train']:7.4f} | {r['cv_mean']:5.4f}+-{r['cv_std']:.4f} | {r['n_leaf']:7d}")

# Pilih dari CV mean tertinggi; kalau seri, pilih yang daunnya paling sedikit (Occam)
cv_terbaik = max(r["cv_mean"] for r in hasil)
kandidat = [r for r in hasil if abs(r["cv_mean"] - cv_terbaik) < 1e-12]
pemenang = min(kandidat, key=lambda r: r["n_leaf"])
print(f"\nPemenang (dipilih dari CV): max_depth={pemenang['max_depth']}, "
      f"min_samples_leaf={pemenang['min_samples_leaf']} "
      f"-> CV {pemenang['cv_mean']:.4f}+-{pemenang['cv_std']:.4f}, "
      f"{pemenang['n_leaf']} daun")
```

```
 max_depth | min_samples_leaf |   train |   CV mean+-std |  n_leaf
----------------------------------------------------------------
         2 |                1 |  0.9582 | 0.9319+-0.0189 |       4
         2 |                5 |  0.9582 | 0.9275+-0.0215 |       4
         2 |               10 |  0.9495 | 0.9231+-0.0139 |       4
         4 |                1 |  0.9868 | 0.9297+-0.0149 |      11
         4 |                5 |  0.9758 | 0.9319+-0.0162 |      10
         4 |               10 |  0.9626 | 0.9143+-0.0254 |       9
         6 |                1 |  0.9978 | 0.9253+-0.0128 |      18
         6 |                5 |  0.9758 | 0.9275+-0.0226 |      13
         6 |               10 |  0.9626 | 0.9143+-0.0254 |      10
      None |                1 |  1.0000 | 0.9165+-0.0179 |      19
      None |                5 |  0.9758 | 0.9275+-0.0226 |      13
      None |               10 |  0.9626 | 0.9143+-0.0254 |      10
```

```
Pemenang (dipilih dari CV): max_depth=2, min_samples_leaf=1 -> CV 0.9319+-0.0189, 4 daun
```

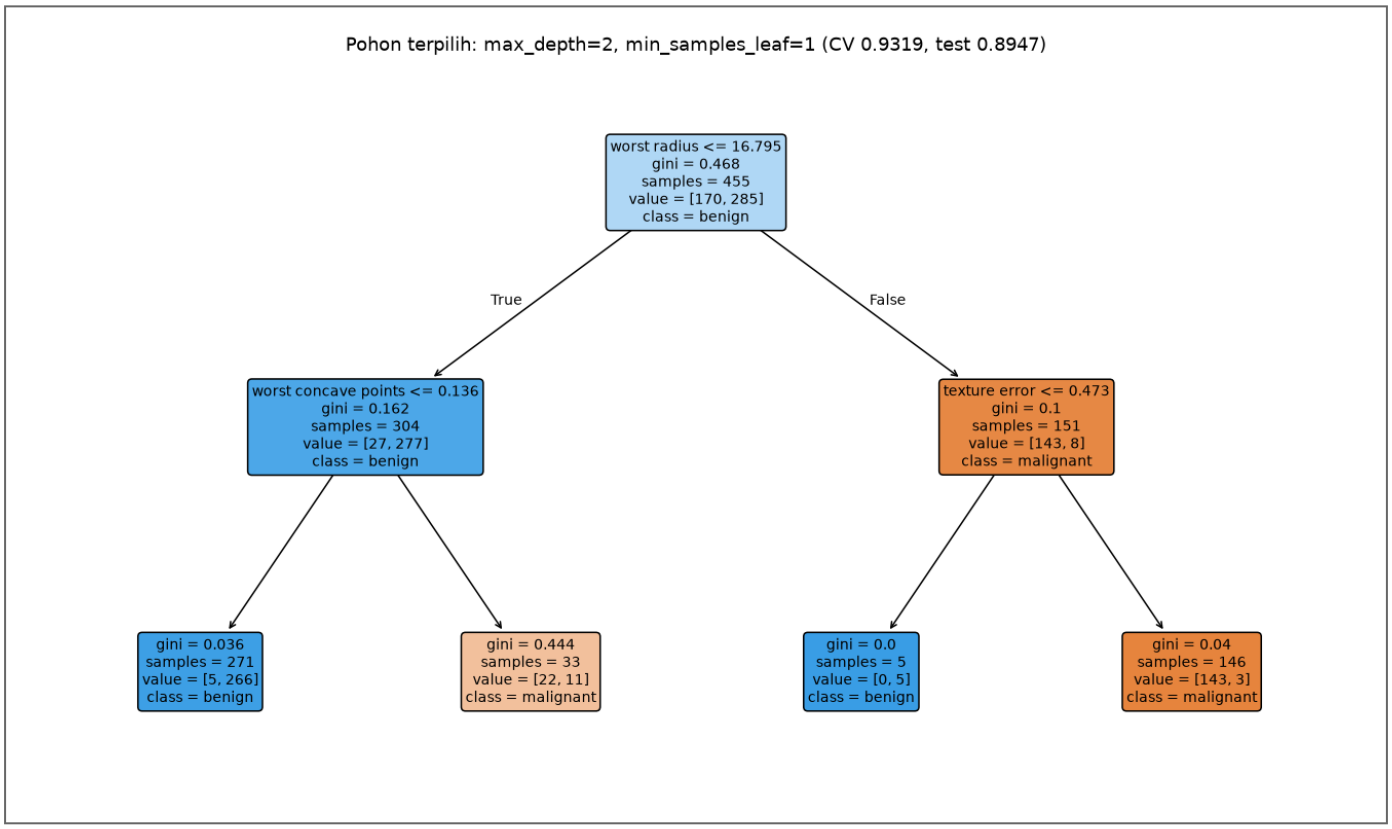
Penjelasan: setiap kombinasi dicoba dengan 5-fold cross-validation di data latih: data latih diputar 5 kali menjadi bagian latih-validasi yang berbeda, lalu skornya dirata-rata. Kolom CV inilah yang dipakai memilih, bukan kolom test. Pemenangnya **max_depth=2, min_samples_leaf=1**: CV tertinggi (0,9319) dengan pohon paling sederhana (4 daun, sedalam 2). Kombinasi (4, 5) CV-nya sama persis tetapi 10 daun, jadi kalah oleh prinsip Occam: kalau skor sama, pilih yang paling kecil.

```
# Langkah 5: evaluasi final pemenang di data test ( dipakai SEKALI di akhir )
pohon_terpilih = DecisionTreeClassifier(
    max_depth=pemenang["max_depth"],
    min_samples_leaf=pemenang["min_samples_leaf"],
    random_state=42)
pohon_terpilih.fit(X_train, y_train)

acc_train = accuracy_score(y_train, pohon_terpilih.predict(X_train))
acc_test = accuracy_score(y_test, pohon_terpilih.predict(X_test))
print(f"Akurasi train       : {acc_train:.4f}")
print(f"Akurasi CV (5-fold) : {pemenang['cv_mean']:.4f} +- {pemenang['cv_std']:.4f}")
print(f"Akurasi test final  : {acc_test:.4f}")
print(f"Gap train-test      : {acc_train - acc_test:.4f}")
print(f"Kedalaman pohon     : {pohon_terpilih.get_depth()}")
print(f"Jumlah leaf node    : {pohon_terpilih.get_n_leaves()}")
```

```
Akurasi train       : 0.9582
Akurasi CV (5-fold) : 0.9319 +- 0.0189
Akurasi test final  : 0.8947
Gap train-test      : 0.0635
Kedalaman pohon     : 2
Jumlah leaf node    : 4
```

```
# Visualisasi pohon terpilih (Latihan 3)
plt.figure(figsize=(12, 7))
plot_tree(pohon_terpilih, feature_names=data.feature_names,
          class_names=data.target_names, filled=True, rounded=True, fontsize=9)
plt.title(f"Pohon terpilih: max_depth={pemenang['max_depth']}, "
          f"min_samples_leaf={pemenang['min_samples_leaf']} "
          f"(CV {pemenang['cv_mean']:.4f}, test {acc_test:.4f})")
plt.tight_layout()
plt.show()
```



Gambar 5.4: Pohon terpilih Latihan 3 (max_depth=2, min_samples_leaf=1): 4 daun, CV 0,9319, test 0,8947.

Interpretasi Latihan 3:

- **Jawaban pertanyaan analisis:** pohon dengan train tertinggi (di sini max_depth=None, min_samples_leaf=1 dengan train **1,0000**) justru bukan yang terbaik, karena ia menghafal data latih sampai ke noise-nya (overfitting). Buktinya skor CV-nya cuma **0,9165**, termasuk paling rendah. Train mengukur seberapa jago model menghafal, CV mengukur seberapa jago model menebak data baru (generalisasi).
- Pemenang dari CV, **max_depth=2, min_samples_leaf=1**, adalah pohon paling sederhana di grid (sedalam 2, 4 daun) dan justru CV-nya tertinggi (0,9319). Sederhana tetapi tetap akurat, sesuai judul praktikumnya.
- Evaluasi final **sekali** di test: akurasi test **0,8947**, gap train-test **0,0635**. Karena test tidak dipakai untuk memilih, angka ini jujur.
- Bandingkan dengan Latihan 1 yang memilih dari skor test: pemenangnya (4, 10) dengan test 0,9474. Kelihatan lebih bagus, tetapi menipu: test dipakai 12 kali untuk memilih, sehingga skornya bias optimistis (test-set overfitting). Pendekatan CV lebih benar karena pemilihan hyperparameter tidak pernah menyentuh data test.

Selesai. Selamat belajar!